# ChaosArena: Multi-Agent Concurrency Testing of REST APIs from Natural-Language Specifications

Hao Wu
Northeastern University
Khoury College of Computer Sciences
Vancouver, Canada
wu.hao11@northeastern.edu

Yvonne Coady*
Northeastern University
Khoury College of Computer Sciences
Vancouver, Canada

## Abstract

Concurrency bugs in REST APIs, including lost updates and unhandled constraint violations, often require overlapping requests from multiple sessions. General-purpose black-box API testing tools rarely exercise such conditions systematically. Specification-guided fuzzers such as RESTler [1] and AutoRestTest [2] generate stateful request sequences, but their published designs do not provide synchronized request-level concurrency. Tool-equipped LLM agents can reason about endpoint semantics and business logic, yet concurrency testing remains dependent on ad-hoc scripts generated during exploration.

We present ChaosArena, a natural-language-guided multi-agent framework for black-box REST API testing. From a natural-language service description, an orchestrator generates required scenarios across six bug categories and assigns them to parallel executors. Executors use independent authenticated sessions and two concurrency tools, `race_pair` and `barrier_concurrent`, to issue barrier-synchronized requests. API discovery results are shared through a cached prompt prefix to reduce repeated probing.

We evaluate ChaosArena on three services from the Web Fuzzing Dataset [17], using Claude Code with the same Sonnet 4.6 model and AutoRestTest [2] as baselines. Among 84 unique findings confirmed across the evaluated runs, ChaosArena identified 59, including 11 concurrency bugs that were not found by either baseline. Claude Code identified 43 findings, while AutoRestTest triggered many server errors but produced no confirmed semantic bug reports.

## 1 Introduction and Related Work

### 1.1 Motivation

Concurrency bugs in REST APIs are a distinct class of defect that cannot be detected by testing one request at a time. A lost-update bug in a shopping cart, for example, may appear only when two add-to-cart requests overlap during the same read–modify–write cycle. An unhandled uniqueness constraint violation, where concurrent duplicate registrations both return HTTP 500 with leaked stack traces instead of a proper 409 Conflict response, requires two independent sessions issuing the same POST simultaneously. Such bugs can arise when shared state is updated without sufficient transaction isolation or application-level coordination. Sequential requests do not reproduce the relevant interleavings.

REST API testing tools must address three concerns: authentication (managing login flows, session tokens, multi-user state), semantic correctness (verifying business logic, status codes, data integrity), and concurrency (detecting race conditions). Existing tools cover at most two of these three.

### 1.2 Existing Approaches

**Specification-guided fuzzers.** RESTler [1] models REST API testing as a state-machine exploration: operations are chained by OpenAPI-declared producer–consumer dependencies, and fuzzing strategies generate payloads to maximize endpoint coverage. RESTler's stateful sequences can reach deep API states, but each sequence executes one request at a time within a single session. AutoRestTest [2] adds multi-agent reinforcement learning (MARL) to prioritize operations that yield new server-side states, achieving higher operation coverage than random fuzzing on complex APIs. However, its reinforcement signal is based on HTTP status code novelty rather than semantic correctness. EvoMaster [3] applies evolutionary search with both white-box (instrumented) and black-box modes, generating test suites that maximize code coverage or fault detection. RESTSpecIT [9] uses LLM-assisted request mutations to improve schema-level coverage without requiring manual test authoring. These tools can generate broad request coverage, but they do not expose concurrency-specific synchronization primitives or require barrier-synchronized multi-session tests in their published designs. Many schema-driven fuzzers also rely on preconfigured credentials rather than modeling coordinated multi-user authorization scenarios.

**LLM-based testing.** LogiAgent [4] introduces a three-agent architecture (scenario generator, executor, validator) augmented with execution memory that accumulates knowledge across test runs. It shows the value of LLM-based semantic testing, but does not establish systematic multi-user authorization or synchronized concurrency testing. RBCTest [5] mines response-body constraints from OpenAPI specifications using LLMs, addressing oracle generation rather than active bug discovery. SAINT [6] combines static analysis with an agentic LLM loop for integration testing, achieving high statement coverage, but it requires source code access and cannot be applied to black-box APIs. SWE-agent [15] demonstrates that tool-equipped LLM agents can effectively use computer interfaces (terminal, editor, browser) for software engineering tasks, establishing the agent–computer interface paradigm that ChaosArena builds upon.

Tool-equipped LLM agents such as Claude Code [16] can interpret endpoint semantics, craft valid payloads across multi-step flows, and reason about expected behavior—capabilities that fuzzers lack. However, they operate in a sequential tool-call loop: the model issues one tool call, waits for its result, then decides the next action. Claude Code can in principle write ad-hoc concurrent scripts,

*Advisor

but this depends on the model independently deciding to attempt concurrency and does not provide barrier synchronization.

**Concurrency testing.** MicroRacer [7] detects concurrency bugs in microservices by instrumenting runtime libraries to intercept and reorder inter-service calls, exposing race conditions through controlled scheduling. It requires source code and library-level instrumentation hooks, making it inapplicable to black-box testing. Semantic Web Racer [8] performs black-box business-logic race detection for web applications by defining race patterns (e.g., time-of-check-to-time-of-use) and replaying request sequences with controlled timing. However, it relies on manually defined race patterns. Neither tool integrates concurrent request generation with LLM-based semantic reasoning about which endpoints are likely to exhibit race conditions.

**The gap.** To the best of our knowledge, prior work has not combined LLM-based semantic reasoning with barrier-synchronized requests issued through multiple authenticated sessions in a black-box REST API testing workflow. Table 1 summarizes the capabilities most relevant to this study.

**Table 1: Comparison of REST API testing tools.**

| Tool | Paradigm | Spec | Auth | Conc. | Sem. |
|---|---|---|---|---|---|
| RESTler [1] | Fuzzer | OAS | Preconf. | No | No |
| EvoMaster [3] | Evol. search | OAS | Preconf. | No | No |
| AutoRestTest [2] | RL fuzzer | OAS | Preconf. | No | Part. |
| RESTSpecIT [9] | LLM mutation | OAS | N/A | No | Part. |
| LogiAgent [4] | Multi-agent | OAS | N/A | No | Yes |
| RBCTest [5] | LLM oracle | OAS | N/A | No | Yes |
| SAINT [6] | Agentic LLM | Src | WB | No | Yes |
| MicroRacer [7] | Instrument. | Src | WB | Yes | No |
| Claude Code [16] | LLM agent | OAS | Probe | Ad-hoc† | Yes |
| **ChaosArena** | **Multi-agent** | **NL** | **Multi** | **Yes** | **Yes** |

†Claude Code can write ad-hoc concurrent scripts (e.g., shell backgrounding), but without barrier synchronization. It attempted this on 1 of 3 services.
OAS = OpenAPI Spec; NL = Natural-language description; Src = Source code; WB = White-box; Preconf. = Preconfigured credentials; Conc. = Concurrent requests; Sem. = Semantic reasoning; Part. = Partial.

## 1.3 ChaosArena and Contributions

ChaosArena addresses the concurrency gap by combining: (1) an orchestrator that derives required test scenarios from a natural-language service description with mandatory concurrency coverage; (2) batch-executor agents with pre-authenticated multi-user sessions; and (3) OS-level concurrency primitives exposed as first-class agent tools. A shared context mechanism based on Anthropic's prompt caching propagates API discovery results to all executors, avoiding redundant probing and improving cost efficiency.

This paper makes four contributions:

(1) A multi-agent architecture with shared context propagation that separates orchestration, batch execution, and concurrent-request tooling.
(2) `race_pair` and `barrier_concurrent` primitives providing barrier-synchronized concurrent requests within an LLM agent loop.
(3) An empirical evaluation on three Web Fuzzing Dataset [17] services comparing ChaosArena against a fuzzer and a tool-equipped LLM agent using the same underlying model.
(4) Confirmation of 11 concurrency bugs—lost updates, constraint violations, race-triggered data leaks—found exclusively by ChaosArena.

# 2 System Design

## 2.1 Overview

ChaosArena operates as a two-phase pipeline. In the required coverage phase, an orchestrator agent reads the service's natural-language specification, derives a ranked list of required test scenarios (Rs), and dispatches them in batches to parallel batch-executor agents. Each executor has pre-authenticated HTTP sessions and domain-specific tools including concurrency primitives. In the exploration phase, a dedicated executor runs open-ended testing beyond the specified scenarios. A verdict drafter synthesizes all findings into a structured report. Figure 1 illustrates the architecture.

## 2.2 Shared Context and Cost Efficiency

A key design decision is how to share API discovery results across executors without redundant probing. Before dispatching any test batches, the orchestrator runs an API probe agent that issues exploratory HTTP requests to discover live resource IDs, user credentials, route patterns, and response shapes. The probe produces a structured markdown playbook containing: authentication methods and credentials, an endpoint map with HTTP methods and paths, response field types, and known constraints.

The playbook is injected as a shared system prompt prefix for all batch executors. The shared prefix is configured for prompt caching and reused across subsequent executor conversations. It means the ~2,000-token playbook is transmitted to the API once and reused from cache for all subsequent executor turns. Since batches run in parallel via `ThreadPoolExecutor`, this caching eliminates what would otherwise be $N \times 2{,}000$ redundant input tokens across $N$ batches.

The turn budget is allocated hierarchically:

- 30% of the total budget is reserved for the probe phase.
- The remaining budget is divided among batches using the formula in §2.4.
- At 85% of a batch's turn budget, the executor receives a warning to stop opening new investigations.
- If all assigned Rs are covered, the executor auto-submits its verdict, saving remaining turns.
- A hard timeout produces a TIMEOUT verdict if the budget is exhausted without a verdict.

Sessions are managed via a module-level `SESSIONS` dictionary keyed by session ID (e.g., "alice", "bob"). Each batch executor starts with an empty session pool and authenticates independently using the credentials from the playbook. Sessions are not shared across batches—this isolation prevents cookie/state interference between concurrent test batches.

## 2.3 Required Scenario Generation

The orchestrator reads the service description and generates Rs. Each R specifies: (1) a natural-language description of the scenario, (2) a severity label, and (3) a test type—*sequential* for single-session flows, *race* for scenarios requiring concurrent requests, or *validation* for boundary checks. Rs are drawn from six categories: concurrency, authentication/authorization, IDOR, input validation, HTTP semantics, and business logic.

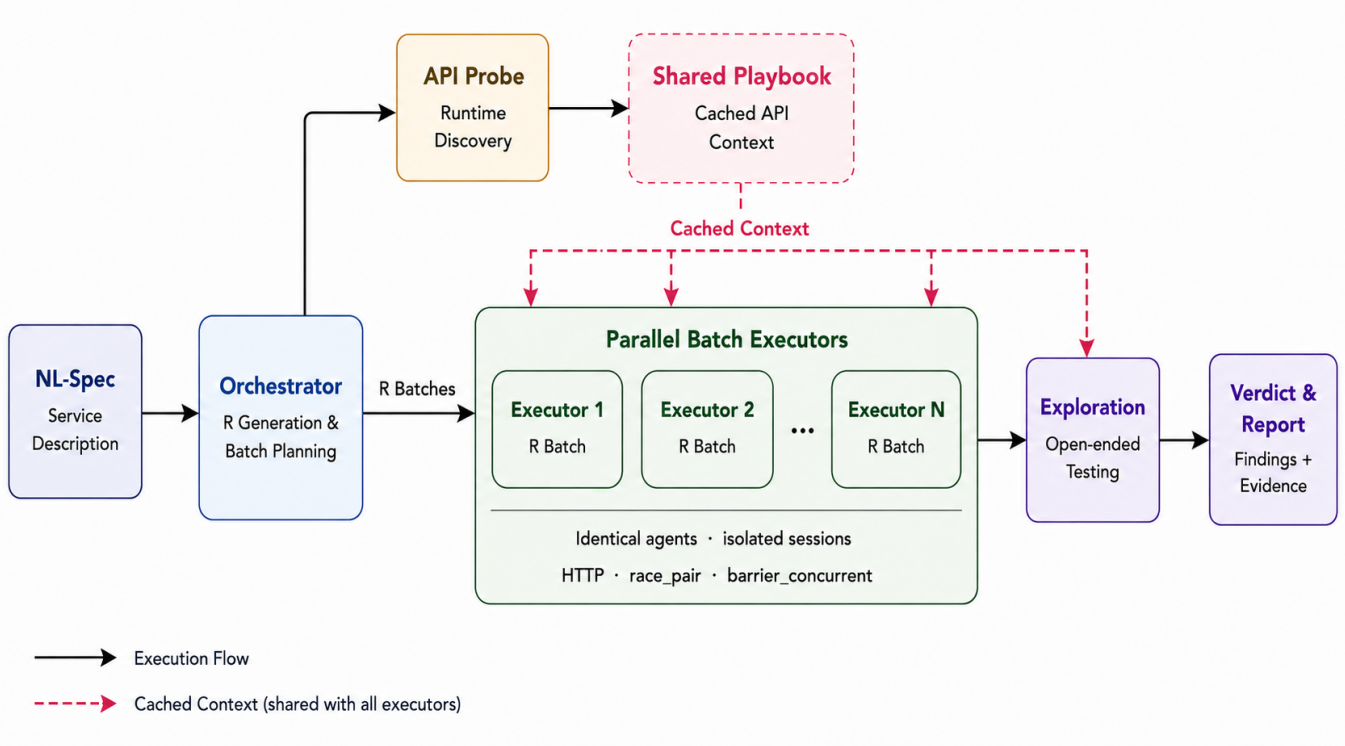


**Figure 1: ChaosArena architecture. The orchestrator generates and batches required scenarios from an NL-Spec. An API probe produces a cached playbook shared across required batch executors. After required batches finish, the orchestrator checks coverage and remaining turn budget; if all Rs are covered, a dedicated exploration executor runs open-ended testing before all outputs are aggregated into the final report.**

**Prompt design.** The orchestrator prompt follows four principles:

(1) **Multi-stage decomposition.** The prompt instructs the model to first enumerate all operations that mutate shared state, then enumerate authentication boundaries, then enumerate edge cases, before drafting scenarios. This follows the multi-stage approach shown to improve test coverage by Ryan et al. [10].

(2) **Mandatory category injection.** Research on LLM-generated code bugs shows that default LLM output under-covers concurrency—Tambon et al. [12] found none of their 10 LLM bug categories includes concurrency. The prompt requires Rs in all six categories; empty categories must include explicit justification.

(3) **Structured chain-of-thought.** The prompt uses a `<thinking>` block where the model enumerates mutation points, async boundaries, and actor/role combinations before emitting Rs. Jain and Purandare [11] showed that LLMs can reason about data races when explicitly prompted to, but will not do so by default.

(4) **Self-review pass.** Following the evaluator-optimizer pattern [13], the prompt includes a self-review step: after generating Rs, the model checks whether all categories are covered and whether any enumerated operation lacks a corresponding R.

The R set is fixed before any executor runs, providing a deterministic coverage contract.

## 2.4 Orchestrator and Batch Planning

Rs are assigned to batches under two rules. First, any R classified as *race* is placed in its own isolated batch to prevent session interference. Second, non-race Rs are grouped by estimated turn budget:

$$\text{max\_turns} = \max\left(6,\ \lceil \hat{t} \times 1.5 \rceil + \text{AUTH\_OVERHEAD} + 2\right)$$

where $\hat{t}$ is the sum of per-R turn estimates and AUTH_OVERHEAD = 2 accounts for re-authentication. A repair mode uses a multiplier of 2.0 for retries.

The orchestrator dispatches batches, waits for each `batch_result` JSON, and updates a coverage map. If any Rs remain UNTESTABLE after the initial pass, repair batches are dispatched with access to prior batch summaries.

## 2.5 Batch Executor

Each executor receives a batch of Rs, the cached playbook prefix, and a turn budget. It accesses three tool categories: (1) HTTP tools for issuing requests with explicit headers and body control; (2) concurrency tools (`race_pair`, `barrier_concurrent`) described in

§2.6; and (3) `record_event`, a structured logging tool that requires a unique title per finding and checks against all previously recorded titles, enforcing deduplication at the tool-call level.

Upon completing its Rs or exhausting its budget, the executor returns a `batch_result` with per-R verdicts, HTTP evidence, and token usage.

## 2.6 Concurrency Tooling

Two concurrency primitives are provided:

`race_pair` fires two HTTP actions in parallel using OS-level threads. Both actions are prepared—headers set, body serialized, connection opened—then released within a configurable skew (typically 0–2,000 $\mu$s). Both responses (status codes, bodies, wall-clock timing) are returned together. Each action carries an explicit id, enabling cross-user scenarios (e.g., Alice vs. Bob).

`barrier_concurrent` issues $N$ simultaneous requests from $N$ sessions, enabling scenarios such as concurrent duplicate registration.

The design rationale is that a tool-equipped LLM agent issues tool calls sequentially within a single execution thread. The $\mu$s-level overlap that triggers database-level constraint violations requires OS-level parallelism. `race_pair` and `barrier_concurrent` expose this capability as agent tools—the model reasons about *what* to test concurrently; the tool handles *how* to achieve temporal overlap. In our evaluation, `race_pair` was used for 2-request race scenarios (e.g., market B02, B03; tracking T02) and `barrier_concurrent` for multi-request scenarios (e.g., user-management U02, U03).

## 2.7 Exploration Phase and Verdict Generation

After all Rs are resolved, an exploration executor probes the service with a larger turn budget and no fixed scenario list. The exploration phase surfaces additional findings—data over-exposure, broken endpoints, semantic inconsistencies—but exhibits a high duplication rate (38–67% across services) due to the absence of a global dedup constraint spanning the full session.

The verdict drafter reads all `batch_result` records and exploration findings, emits per-R verdicts with supporting evidence, and produces an aggregate PASS/FAIL verdict.

# 3 Evaluation

## 3.1 Experimental Setup

**Services under test.** We select three Java Spring Boot REST services from the Web Fuzzing Dataset (WFD) [17], a benchmark of 36 open-source REST APIs maintained by Sahin, Zhang, and Arcuri. The three services were chosen because each enforces a different authentication mechanism (Table 2).

Swagger specs were converted to OpenAPI via `swagger2openapi`. All services were deployed locally via Docker Compose and reset to a clean state before each run.

**Input format.** ChaosArena received a natural-language service description for each service, derived from the OpenAPI specification (endpoint listing, parameter types, authentication requirements). This is the framework's native input format—it does not parse OpenAPI schemas directly but uses the LLM to interpret the description.

**Baselines.** AutoRestTest [2] ran with OpenAPI specs and a 1,200-second budget per service. Claude Code [16] was invoked via CLI with the same service information provided for fair comparison. Both ChaosArena and Claude Code used Sonnet 4.6 to isolate the effect of architecture rather than model capability.

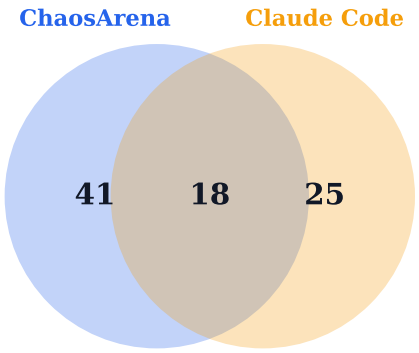


**Figure 2: Bug discovery overlap between ChaosArena and Claude Code.**

**Bug confirmation.** A finding is counted as a confirmed unique bug if: (i) it includes an exact HTTP request and response, (ii) it represents a distinct violation of expected behavior, and (iii) it is not a duplicate within the same tool run. Cross-tool deduplication was performed manually. AutoRestTest produces unstructured HTTP 500 error logs without semantic classification; these raw failures are reported separately, but they are not counted as confirmed unique semantic bugs unless they can be mapped to a distinct behavior violation. All table values below are derived from the normalized result table `data/collected_data.csv`, which combines confirmed bug rows with raw run summaries from ChaosArena, Claude Code, and AutoRestTest.

## 3.2 Bug Counts

Table 3 shows confirmed bugs per tool per service.

**Overlap.** Figure 2 shows the discovery overlap between ChaosArena and Claude Code. Of 84 bugs: 41 were found only by ChaosArena, 25 only by Claude Code, and 18 by both LLM-based tools. The 41 ChaosArena-exclusive bugs include all 11 concurrency bugs plus systematic auth/IDOR findings from spec-driven Rs. Claude Code's 25 exclusive bugs are concentrated in HTTP semantics deviations and deep side-effects.

Only 18 of the 84 confirmed findings were shared by ChaosArena and Claude Code. The observed difference is consistent with their testing strategies: ChaosArena's required scenarios emphasize authorization boundaries and concurrent state changes, whereas Claude Code spends more of its budget on open-ended endpoint exploration. Its exclusive findings include malformed date handling, cascade-deletion side effects, and inconsistent fields between POST and GET responses.

**Note on AutoRestTest.** AutoRestTest is evaluated here as a fuzzer baseline for raw failures and operation coverage, not as a semantic bug reporter. AutoRestTest sent 120,525 requests on market, all of which returned HTTP 500, and did not produce a

**Table 2: Services under test (from WFD [17]).**

| Service | Endpoints | Auth | Stateful surface tested |
|---|---|---|---|
| market | 13 | Basic Auth | Cart, payment, registration, and order ownership |
| tracking | 67 | Cookie session | Assignments, employees, credentials, projects, and departments |
| user-mgmt | 21 | Session + RBAC | User profiles, role assignment, permissions, and account lifecycle |

**Table 3: Confirmed unique bugs per tool per service.**

| Service | $CA_R$ | $CA_E$ | CC | ART | Union |
|---|---|---|---|---|---|
| market | 14 | 6 | 7 | 0 | 23 |
| tracking | 14 | 6 | 23 | 0 | 33 |
| user-mgmt | 14 | 5 | 13 | 0 | 28 |
| **Total** | **42** | **17** | **43** | **0** | **84** |

$CA_R$ = ChaosArena required phase; $CA_E$ = ChaosArena exploration phase (unique findings not already discovered in the required phase); CC = Claude Code; ART = AutoRestTest. Union is the deduplicated total across tools for each service.

confirmed semantic report for that service. On tracking, it processed 68.7% of operations and triggered 37 HTTP 500 responses. On user-management, it processed 90.5% of operations but triggered 8,938 HTTP 500 responses, mostly on permission-update flows. Because it produces unstructured error logs without semantic classification, AutoRestTest is excluded from the category and severity analyses that follow.

## 3.3 Category Coverage

The remaining analysis compares ChaosArena and Claude Code only. Table 4 provides exact category counts. The category distribution differs across the two tools. ChaosArena reports more findings in Concurrency and authorization/security categories, whereas Claude Code reports more in HTTP Semantics.

**Table 4: Bug category coverage (ChaosArena vs. Claude Code).**

| Category | ChaosArena | Claude Code |
|---|---|---|
| **Concurrency** | **11** | **0** |
| Auth / IDOR / Security | 23 | 14 |
| HTTP Semantics | 15 | 24 |
| Business Logic | 3 | 2 |
| Input Validation | 5 | 1 |
| Data Integrity / API Design | 2 | 2 |

ChaosArena found more multi-session and concurrency-related issues (Concurrency: 11 vs. 0; Auth/IDOR/Security: 23 vs. 14). Claude Code found more HTTP-semantics issues (24 vs. 15). A likely contributor to the Auth/IDOR difference is that ChaosArena explicitly provisions multiple user roles and assigns cross-user checks as required scenarios; the baseline did not apply the same authorization matrix systematically.

**Concurrency bugs.** ChaosArena found all 11 concurrency bugs; Claude Code found zero. Table 5 details each bug.

Across the recorded runs, 7 of the 11 concurrency bugs were triggered by `race_pair` and 4 by `barrier_concurrent`; `parallel_n` was used as a helper but did not produce a confirmed concurrency bug. This is not an ablation result, but it indicates that synchronized request release, rather than generic HTTP execution, was the decisive tool capability.

All 11 bugs are HIGH severity under our impact-based reclassification. Seven involve unhandled database constraint violations that leak internal stack traces (Hibernate class names, SQL constraint identifiers). The remaining four involve silent data corruption (lost updates, stale state). These bugs fall into two distinct failure modes. The first is silent data corruption: in B01, two simultaneous add-to-cart requests for the same product both read the current quantity as 0, both write quantity=1, and the second write overwrites the first—the customer adds two items but only one is persisted. The second is unhandled constraint violations: in U01 through U04, concurrent duplicate writes trigger database uniqueness constraints, and the application returns HTTP 500 with raw `ConstraintViolationException` stack traces instead of a proper 409 Conflict response. Neither failure mode was reproduced by the sequential baselines in our runs. The client-side synchronization tools increased the likelihood that the corresponding server-side operations overlapped, although we did not instrument the services to measure the exact overlap window.

The concurrency findings span different endpoint and operation pairs. Market findings involve cart and payment state transitions; tracking findings involve assignment lifecycle operations; and user-management findings involve registration and role-management flows. This spread supports testing a range of shared-state mutations rather than focusing on a single predefined race pattern.

**Other categories.** ChaosArena leads on Auth/IDOR/Security (23 vs. 14), due to its pre-authenticated multi-user sessions enabling systematic cross-user access testing. Claude Code leads on HTTP Semantics (24 vs. 15), due to its unconstrained exploratory approach testing more endpoint/status-code combinations. The per-service breakdown (Figure 3) reveals that the category distribution is not uniform across services: market and user-management are dominated by Auth/IDOR and Concurrency findings, reflecting their richer authorization models, while tracking—with 67 endpoints—yields a broader spread across HTTP Semantics and Business Logic. This suggests that ChaosArena's required-scenario generation adapts its emphasis to each service's authentication complexity and endpoint surface area.

**Severity distribution.** Figure 4 shows the severity breakdown. We reclassified severity from the confirmed bug descriptions rather than using the parenthetical labels in ChaosArena verdict headings, which denote scenario confidence rather than bug impact. Our rubric treats unauthorized sensitive-data access, privilege escalation, and destructive account actions as CRITICAL; race-induced data corruption, unhandled internal exceptions, and serious state

**Table 5: All 11 concurrency bugs found by ChaosArena. All are exclusive—neither baseline detected any of them.**

| ID | Service | Race Pattern | Tool Used | Observed Failure | Severity |
|---|---|---|---|---|---|
| B01 | market | Concurrent add-to-cart (same product) | `race_pair` | Lost update: qty=1 not 2 | HIGH |
| B02 | market | Concurrent pay + add-item | `race_pair` | Stale cart not cleared | HIGH |
| B03 | market | Concurrent clear-cart + pay | `race_pair` | Hibernate StaleStateException | HIGH |
| B04 | market | Concurrent duplicate registration | `race_pair` | 500 NonUniqueResultException | HIGH |
| T01 | tracking | Concurrent duplicate POST assignment | `race_pair` | 400 + SQL constraint leak | HIGH |
| T02 | tracking | Concurrent DELETE same assignment | `race_pair` | StaleStateException exposed | HIGH |
| T03 | tracking | Concurrent PUT (INSERT not UPDATE) | `race_pair` | PK conflict → 400 | HIGH |
| U01 | user-mgmt | Concurrent duplicate registration | `barrier_conc.` | 500 ConstraintViolation | HIGH |
| U02 | user-mgmt | Concurrent role assign + delete | `barrier_conc.` | 500 ConstraintViolation | HIGH |
| U03 | user-mgmt | Concurrent permission delete + assign | `barrier_conc.` | 500 ConstraintViolation | HIGH |
| U04 | user-mgmt | Concurrent duplicate admin creation | `barrier_conc.` | 500 ConstraintViolation | HIGH |

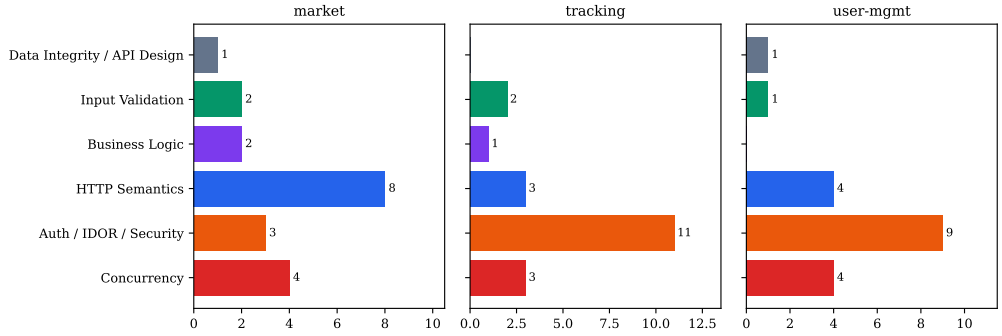


**Figure 3: ChaosArena bug categories per service. Market and user-management are dominated by Auth/IDOR and Concurrency findings; tracking shows a broader category spread reflecting its larger endpoint surface.**

inconsistency as HIGH; and status-code, schema, or validation deviations with limited security impact as MEDIUM or LOW. Under this impact-based mapping, ChaosArena reports 17 CRITICAL, 20 HIGH, 20 MEDIUM, and 2 LOW findings. Claude Code reports 11 CRITICAL, 11 HIGH, 10 MEDIUM, and 11 LOW findings under the same mapping.

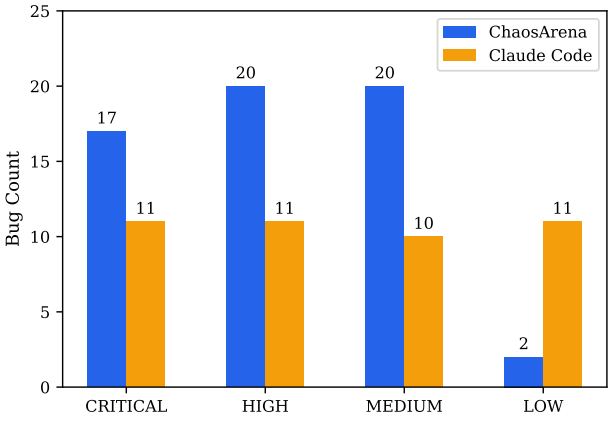


**Figure 4: Severity distribution: ChaosArena vs. Claude Code.**

## 3.4 Cost and Efficiency

**Table 6: Cost and efficiency by tool and phase.**

| Tool | Phase | Cost | Duration | Bugs | Bugs/$ |
|---|---|---|---|---|---|
| CA | Required | $8.11 | — | 42 | 5.18 |
| CA | Exploration | $3.34 | — | 17 | 5.08 |
| CA | **Total** | **$11.85** | **6.85 min** | **59** | **4.98** |
| CC | (single phase) | $3.70 | 8.5 min | 43 | 11.62 |

CA = ChaosArena; CC = Claude Code. Costs and bugs are totals across all 3 services. CA phase costs are agent execution costs; CA Total uses the verdict-reported total cost, including drafter cost. CA duration is the average wall-clock time per service computed from verdict Started/Finished timestamps. CA Required includes orchestrator planning, API probe, and all R-batches.

Claude Code achieves higher bugs-per-dollar (11.62 vs. 4.98) because it operates without multi-session management overhead. However, ChaosArena's required phase nearly matches Claude Code's total confirmed bug count (42 vs. 43) and includes all 11 concurrency bugs that neither baseline detected. The exploration phase contributes 17 additional unique findings at a comparable per-dollar rate (5.08), but also produces substantial duplication: 38–67% of exploratory findings are near-identical restatements of required-phase discoveries.

Claude Code averaged 8.5 minutes per service in the recorded runs. ChaosArena averaged 6.85 minutes per service, computed from verdict start/end times (3.34 minutes for market, 7.88 for tracking, and 9.33 for user-management). This lower wall-clock time is consistent with ChaosArena's parallel batch execution model, in which runtime is bounded by the slowest batch rather than the total number of scenarios.

The main inefficiency in ChaosArena is duplicate discovery across independent batches. These duplicates do not change the deduplicated final bug count, but they do consume tokens in both executor runs and verdict drafting. A practical mitigation is to maintain a shared finding registry and inject it into later batch prompts as a blocklist.

# 4 Discussion

## 4.1 Structured vs. Ad-Hoc Concurrency Testing

Claude Code attempted concurrent testing on one of three services (tracking), using a shell `for` loop with background execution (`&`) and `wait` to fire 5 simultaneous POST requests to the departments endpoint. The requests succeeded—5 departments were created with sequential IDs—but Claude Code did not identify this as a concurrency bug or investigate further race patterns.

The attempt shows that ad-hoc concurrent scripting is possible, but its use depends on the agent's exploration choices and on how the script synchronizes requests. Claude Code can issue concurrent requests by writing temporary scripts, but: (1) it attempted this on only 1 of 3 services; (2) the shell backgrounding provides no barrier synchronization—requests start at arbitrary intervals rather than within $\mu$s skew; (3) it tested one pattern (duplicate creation) versus ChaosArena's systematic coverage of 4–7 race scenarios per service; (4) it operated with a single user session, precluding cross-user race scenarios (e.g., Alice vs. Bob IDOR).

ChaosArena makes concurrency testing an explicit part of the workflow rather than leaving it to an agent's discretionary use of temporary scripts. The R coverage contract ensures that concurrency scenarios are tested on every service, regardless of whether the model would independently decide to attempt them.

## 4.2 Complementarity

The limited overlap in Figure 2 suggests that the two LLM-based workflows are complementary. ChaosArena contributes structured concurrency and authorization coverage, while Claude Code contributes unconstrained exploration of HTTP semantics and side effects. A practical pipeline could therefore run ChaosArena for required-scenario coverage and race testing, then run an unconstrained LLM agent for exploratory edge cases.

## 4.3 Scalability and Generalizability

ChaosArena's batch-based architecture is designed to scale to larger APIs. The orchestrator partitions Rs into independent batches, each with its own turn budget and session pool, executed in parallel via `ThreadPoolExecutor`. As the number of endpoints grows, the orchestrator generates more Rs and dispatches more batches, with wall-clock time bounded by the slowest batch rather than the total number of scenarios. In our evaluation, the three services ranged from 13 to 67 endpoints; the turn budget formula (§2.4) adapted automatically, producing 6–12 batches per service.

The concurrency primitives (`race_pair`, `barrier_concurrent`) are transport-level tools that operate on arbitrary HTTP requests. They are agnostic to the application framework, database backend, or programming language of the service under test. The 11 concurrency bugs found in our evaluation span two distinct failure modes (silent data corruption and unhandled constraint violations) across three different authentication mechanisms (Basic Auth, cookie sessions, session+RBAC), suggesting that the approach generalizes across common web application patterns.

Several factors limit generalizability to production environments. First, production APIs often use OAuth 2.0 or multi-factor authentication, which requires more complex session establishment than the single-step flows in our test services. The playbook mechanism can represent these flows, but the probe agent must be capable of completing them. Second, distributed architectures (microservices, message queues, eventual consistency) introduce concurrency patterns at the inter-service level that cannot be triggered by concurrent HTTP requests to a single API gateway. MicroRacer [7] addresses this through runtime instrumentation, which ChaosArena's black-box approach cannot replicate. Third, our evaluation uses demo-grade services where bugs are relatively dense; production services with lower bug density may require larger turn budgets and more targeted R generation.

## 4.4 Limitations and Threats to Validity

**Spec dependency.** ChaosArena accepts natural-language service descriptions as input, which avoids a hard dependency on any particular specification format. However, the quality of generated Rs depends on how completely the description covers the API's endpoints, parameters, and authentication flows. In this evaluation, descriptions were derived from OpenAPI specs; services lacking any documentation would require manual description authoring or an extended probe phase.

**Evaluation scope.** Three open-source, demo-grade services from WFD. Production services have larger APIs, more complex auth flows (OAuth, MFA), and different concurrency patterns.

**Exploration duplication.** 38–67% of exploratory findings were near-identical restatements of required-phase findings, representing token waste. Passing accumulated finding titles as a blocklist would reduce this.

**Manual deduplication.** Cross-tool deduplication was manual. The Concurrency row (11/0/0) is unambiguous, but overlap counts for other categories could change with different criteria.

**Reproducibility.** LLM outputs are non-deterministic. The R coverage contract ensures all specified categories are exercised, but exact bug counts may vary across runs.

# 5 Conclusion

Among the 84 unique findings confirmed across all evaluated runs, ChaosArena reported 59, including all 11 concurrency bugs. Both baselines—an RL-guided fuzzer and a tool-equipped LLM agent on the same Sonnet 4.6 model—find zero concurrency bugs. The main difference observed in this evaluation concerns coverage rather than bug count alone. A sequential tool interface can still launch

concurrent scripts, but it does not standardize synchronization or require concurrency scenarios to be exercised. ChaosArena supplies both the testing objective and the request primitives.

Our results suggest that synchronized request tools can make race-oriented tests practical within a conventional LLM agent loop. In the evaluated services, combining these tools with required scenario generation produced concurrency findings that the baselines did not report. The shared playbook and parallel batch execution reduce repeated discovery work in the architecture, although a larger evaluation is needed to separate the effects of caching, batching, and concurrency-specific tooling.

Future work includes: (i) exploration-phase deduplication via accumulated finding blocklists; (ii) evaluation on production-grade services with OAuth/MFA; (iii) adaptive turn-budget estimation from observed execution traces; (iv) automated service description generation from live API probing for undocumented services.

## References


[1] V. Atlidakis, P. Godefroid, and M. Polishchuk. RESTler: Stateful REST API Fuzzing. In *Proc. ICSE*, 2019.

[2] M. Kim, T. Stennett, S. Sinha, and A. Orso. AutoRestTest: A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs. *arXiv:2411.07098*, 2024. ICSE 2025.

[3] A. Arcuri. RESTful API Automated Test Case Generation with EvoMaster. *ACM TOSEM*, 2019.

[4] K. Zhang et al. LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents. *arXiv:2503.15079*, 2025.

[5] K. Huynh et al. RBCTest: Leveraging LLMs to Mine and Verify Oracles of API Response Bodies for RESTful API Testing. *arXiv:2504.17287*, 2025. ICSE 2026.

[6] R. Pan et al. SAINT: Service-level Integration Test Generation with Program Analysis and LLM-based Agents. *arXiv:2511.13305*, 2025. ICSE 2026.

[7] Zhiling Deng, Juepeng Wang, Zhuangbin Chen. MicroRacer: Detecting Concurrency Bugs for Cloud Service Systems. *arXiv:2512.05716*, 2025.

[8] M. Alidoosti, A. Nowroozi, and A. Nickabadi. Semantic Web Application Race Condition Detection. *Expert Systems with Applications*, Elsevier, 2022.

[9] A. Decrop et al. RESTSpecIT: Automated REST API Documentation and Testing via LLM-Assisted Request Mutations. *arXiv:2402.05102*, 2024.

[10] H. Ryan et al. Code-Aware Prompting: Coverage-Guided Test Generation using LLM (SymPrompt). *arXiv:2402.00097*, 2024.

[11] R. Jain and A. Purandare. Assessing LLMs in Comprehending and Verifying Concurrent Programs across Memory Models. *arXiv:2501.14326*, 2025.

[12] F. Tambon et al. Bugs in Large Language Models Generated Code: An Empirical Study. *arXiv:2403.08937*, 2024.

[13] Erik S. and Barry Zhang. Building Effective Agents. Anthropic Engineering Blog, December 2024.

[14] B. Willard and R. Louf. Efficient Guided Generation for Large Language Models (Outlines). *arXiv:2307.09702*, 2023.

[15] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv:2405.15793*, 2024. NeurIPS 2024.

[16] Anthropic. Claude Code. https://docs.anthropic.com/en/docs/claude-code, 2025.

[17] O. Sahin, M. Zhang, and A. Arcuri. WFC/WFD: Web Fuzzing Commons, Dataset and Guidelines. *arXiv:2509.01612*, 2025.